Multimodal Graph Learning Framework for Mortality and Sepsis Prediction using eICU

PROJECT REPORT [Phase-I]

Submitted in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

Department of Computer Science and Engineering

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)



Submitted by

Rudransh Pratap Singh (24U020009 Semester - V)
Aryan Dwivedi (24U020064 Semester - V)

Under the supervision of

Dr Supriyo Mandal
Computer Science and Engineering
Novemeber 2026

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

CERTIFICATE

This is to certify that the work embodied in this report entitled **"Multimodal Graph Learning Framework for Mortality and Sepsis Prediction using eICU"** has been satisfactorily completed by **Rudransh Pratap Singh (24U020009) and Aryan Dwivedi (24U020064)** It is a bona fide piece of work, carried out under our guidance in the **Department of Computer Science and Engineering**, **Indian Institute of Information Technology, Bhopal** for the partial fulfillment of the Bachelor of Engineering during the academic year 2026.

Date:

Dr Supriyo Mandal,
Department of CSE
Department of CSE
IIIT Bhopal (M.P.)

Dr. Rahul Chaurasia
IIIT Bhopal (M.P.)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

DECLARATION

We hereby declare that the following major project synopsis entitled **"Multimodal Graph Learning Framework for Mortality and Sepsis Prediction using eICU"** presented in the partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in** Computer Science and Engineering. It is an authentic documentation of our original work carried out under the able guidance of **Dr Supriyo Mandal.** The work has been carried out entirely at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole to award of any degree or professional diploma in any other institute or organization.

We, with this, declare that the facts mentioned above are true to the best of our knowledge. In case of any unlikely discrepancy that may occur, we will be the ones to take responsibility.

Rudransh Pratap Singh (24U020009)

Aryan Dwivedi (24U020064)

ACKNOWLEDGEMENT

We, Rudransh Pratap Singh and Aryan Dwivedi, would like to express our sincere gratitude to everyone who supported and guided us throughout the successful completion of our minor project titled "Multimodal Graph Learning Framework of Mortality and Sepsis Prediction"
.
First and foremost, we would like to extend our heartfelt thanks to our esteemed **Director**, Dr Ashutosh Kumar Singh, for providing us with the necessary resources and facilities for conducting our project. His encouragement and vision played a key role in helping us accomplish our goals.

We are deeply grateful to our **Project Guide**, Dr Supriyo Mandal, for his constant guidance, invaluable suggestions, and expert advice throughout the project. His knowledge and insights helped us enhance the quality of our work, and his support kept us motivated during difficult times.

Our sincere thanks to the **Department Coordinator**, Dr Rahul Chaurasia, for the valuable support and encouragement provided during completion of the project.

We would also like to express our gratitude to all the **faculties** of the Department of Computer Science Engineering for their valuable guidance and support in clarifying doubts and providing us with the right direction throughout the smooth execution of our project.

A special thanks to our **friends** for their continuous support, valuable suggestions, and helping us during various stages of the project. Their insights and encouragement motivated us to keep working on the project with enthusiasm.

Last but certainly not the least, we would like to express our deepest gratitude to our **parents** and family members for their love, patience, and continuous moral support throughout the course of this project. Their understanding and encouragement were the driving force behind our efforts.

We are truly thankful to everyone who directly or indirectly contributed to the success of this project. Without their assistance and support, this project would not have been possible.

Rudransh Pratap Singh
24U020009
Aryan Dwivedi
24U020064

AREA OF WORK

Our project will mainly focus on MEDICAL ICU DATA and DEEP LEARNING using a set of tools and technologies that are required for the development of large-scale architectures.

While one may argue that the same thing could have been done using machine learning, that's where the real catch is. As it is already mentioned that the tools and architecture we'll be using mainly focus on large scale deployments.

It's been a fairly long period since humans started using machine learning in medical and then extracting useful insights from the data, but in past decade the immense growth in data has posed a serious challenge for computing that's where deep learning came and with time, we also acquired the capacity to process real data sets and ever since it's just keeps growing bigger and bigger.

In our project we will be looking at architecture of deep learning pipeline, and we will also look at analysis of different architectures.

TABLE OF CONTENT

S.no	Title	Page No.
	Certificate	i
	Declaration	ii
	Abstract	iii
1	Introduction	1
2	Literature Review	4
3	Problem Definition	9
4	Proposed Methodology and Work Description	10
5	Proposed Algorithms	20
6	Proposed Flowchart	22
7	Implementation	25
8	Result Discussion and Analysis	29
9	Conclusion and Future Scope	38
	References	39
	List of Publications	
	Plagiarism Report	42

LIST OF FIGURES

S.NO	Fig No.	Description	Page No.
1	4.1	Data flow of LSTM Model	21
2	4.2	Optimization history and Importance of hyperparameters	22
3	4.3	GRU Data Flow	22
4	4.4	K-NN graph with k = 10 of hospital_73	24
5	4.5	GCN Architecture Overview	26
6	4.6	Oversmoothing of GCN	27
7	6.1	Complete Workflow	31
8	6.2	Hospital- Wise Split flow	32
9	6.3	Flowchart of Patient Similarity Graph Construction	33
10	8.1	Loss by Epoch (left), AUPRC per epoch (right)	38
11	8.2	Classification Curves for LSTM	38
12	8.3	Confusion Matrix of Mortality and Sepsis of LSTM	39
13	8.4	Test Performance across seeds 42, 123, 999	40
14	8.5	Attention Based Fusion heat Map	41
15	8.6	Stratification of Attention	41
16	8.7	Confusion Matrix of GCN	43
17	8.8	ROC Curve and PR Curve for Mortality and Sepsis	44
18	8.9	GCN Total and Task Specific Loss	45
19	8.10	Validation Performance of GCN	45

LIST OF TABLES

S.NO	Table No.	Description	Page No.
1	2.1	Comparison of Different Clinical Databases	14
2	4.1	Hospital Wise Partition	19
3	4.2	Hyperparameters for LSTM Model	20
4	7.1	Comparison of Timeseries Imputation Techniques	35
5	8.1	AUROC and AUPRC of LSTM Model	39
6	8.2	AUROC and AUPRC of GRU Model	40
7	8.3	Comparison of LSTM and GRU	41
8	8.4	AUROC, AUPRC and Accuracy of Different Fusion Techniques	42
9	8.5	Mortality and Sepsis AUPRC of GCN for Different Values of k	43
10	8.6	GCN Evaluation metrics	44
11	8.7	Mortality and Sepsis AUROC and AUPRC at Different Stages	46

ABSTRACT

Prediction of hospital mortality at an early stage is a difficult task in intensive care units (ICUs). Clinical scoring methods help to predict the risks of the patients, but there is a possibility that they fail to consider the dynamic changes in the clinical measures and relationships between the various kinds of patient information. Deep learning can help in discovering such patterns from clinical data. This project aims to examine a multimodal deep learning approach for in-hospital mortality and sepsis prediction.

The experiment uses the eICU-CRD. It is a multi-center critical care database that consists of patient data from hospitals all over the United States. There are two kinds of data used: tabular data and time series data. Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) are used to learn temporal representations, while an MLP produces static embeddings. Different multimodal fusion techniques such as additive, concatenative, gated, and attention-based fusions are used to combine the embeddings. The model performance is mainly measured through Area Under the Receiver Operating Characteristic Curve (AUROC), Area Under the Precision-Recall Curve (AUPRC), F1-score, and Recall.

The temporal LSTM and GRU were implemented with satisfactory results. The multimodal fusion provided improvements in predictive performance. The attention-based fusion technique provided the best results out of all the fusion techniques. A patient similarity graph was made using k-NN and cosine similarity. A Graph Convolution Network (GCN) was implemented to predict the graph embeddings. The results suggest that using tabular clinical information along with temporal information can enhance mortality risk discrimination.

The results indicate the possible use of multi-modal deep learning to predict ICU in-hospital mortality. Static features in addition to time-series can provide a better description of the patient's condition than using only the temporal features. A comparative study of the different approaches and models results in the best existing architecture.

Source Code: https://github.com/ryn379/Multimodal-Graph-Learning-Framework

Keywords: Multimodal Deep Learning, ICU Mortality Prediction, Sepsis Prediction, Temporal Modeling, Attention-Based Fusion, Graph Convolutional Network (GCN).

SECTION 1: INTRODUCTION

Intensive care units (ICU) are specialized departments within hospitals that provide continuous care and high-level medical intervention for critically ill patients. Predicting the early mortality and sepsis is crucial for interventions, better clinical decisions, and optimal use of resources [1, 2]. Current approaches for this are based on clinical severity scores and some statistical approaches that may not be able to effectively capture the complex inter-relation among clinical variables or changes in clinical conditions. In contrast, deep learning approach can capture patterns in large quantity of clinical data. Electronic health record (EHR) systems enable availability of patient demographic, physiological measurements, lab test, medication, treatment, clinical severity score to train models for predictions.

This work investigates deep learning-based approaches to predict in-hospital mortality and sepsis based on eICU collaborative research database (eICU-CRD) [3]. The characteristics of patient and dynamic change in clinical conditions are presented in static table features and time series. Long short-term memory (LSTM) and gated recurrent unit (GRU) are employed to capture the dynamic patterns [4, 5, 6]. MLP is used to learn representations of static features. Various methods for feature fusion are explored to determine if combination of representations can boost the prediction performance [7, 8]. Graph convolutional network (GCN) is also investigated for incorporating the relations among similar patients from clinical perspective.
Due to the lack of rich, orderly and heterogeneous clinical data from ICU, the prediction of accurate outcome is often difficult. The variety of sampling frequencies of laboratory measurements and treatments dynamically changing the heterogeneity of disease progression among different patients could miss critical time series and relational information if we just focus on single measurements or aggregate-level clinical scores. More comprehensive patient representations can be achieved by incorporating multi-modality data. Besides, since eICU-CRD [3] contains the data from multiple hospitals, we can study the model generalizability for different healthcare institutions. The challenges inspire us to build a combination of temporal learning and static feature representation as well as graph-based information aggregation.

The clinical outcome prediction is to predict the probability of occurrence of some medical events based on patient data. There exist some crucial clinical outcomes for critical care, including mortality, sepsis, organ dysfunction and long-term hospital stay [1, 2, 9]. In this project, we pay attention to predict in-hospital mortality and sepsis for early detection of high-risk patients so as to better monitor and allocate resources. Graph-based methods model the patient as the node and connect clinically similar patients with edges. Graph convolutional network (GCN) [10] is introduced to aggregate information from adjacent nodes to learn the representations which contain both individual characteristics and patient relations, leading to potentially better performance in prediction.
The prediction is based on the relevant outcomes from the dataset. The prediction target is obtained from the hospital_expired which is the in-hospital mortality. 0 means survived to discharge, 1 means died during the hospitalization [3]. And, the prediction target is also obtained from the variable, sepsis. 0 means non-sepsis, 1 means sepsis. This leads to binary outcomes

that formulate this problem as a supervised classification problem. And for accurate prediction, the input features should be carefully selected based on the availability at the prediction time. We should not use any information that could reveal future outcomes.

The traditional severity scoring methods in ICU evaluates severity of illness, and predicts risk for patients. The severity of illness and risk of mortality is estimated by the Acute Physiology and Chronic Health Evaluation (APACHE) system which utilizes the physiological and clinical information. Illness severity is measured by the Simplified Acute Physiology Score (SAPS). Dysfunctions of important organs are measured by the Sequential Organ Failure Assessment (SOFA). These provide good clinical assessment. However, the summary of scores cannot capture the complex and temporal interaction among lab findings, medications, infusions, and treatments. We can complement these by using deep learning to learn the patterns from the clinical records. Also, we can use k-nearest neighbor (k-NN) graph to connect the similar patients for the purpose of graph-based learning [11, 12].
The eICU-CRD [3] contains not only APACHE-related variables but also many clinical features which need to be carefully selected to avoid data leakage. We should not include any data that provide clues to the target or information beyond the time of prediction, as it will give misleading results about the performance. Feature selection should be done based on its clinical relevance and availability at prediction time. Proper preprocessing, missing value handling, and temporal alignment are important. Proper splitting of training and testing data is required to get reliable results. These steps will enable a fair evaluation of LSTM [4], GRU, MLP, and graph-based methods on ICU mortality and sepsis prediction.

The 4 major objectives for this project are:

- Clinical Data Extraction and Preprocessing: To extract and organize eICU [3] clinical records hospital-wise, handle missing values, normalize numerical features, remove low-variance or weakly informative features, and construct chronologically ordered patient-specific time-series sequences using appropriate binning, forward filling, and zero padding techniques.
- Temporal and Static Representation Learning: To implement and compare LSTM and GRU architectures [4, 5] for learning temporal patterns from clinical time-series data and use a MLP to transform 67 static patient features into compact 64-dimensional representations .
- Multimodal Fusion and Graph-Based Learning: To combine temporal and static representations using multimodal fusion techniques, construct a patient similarity graph using cosine similarity and k-nearest neighbors within individual hospitals, and apply a two-layer Graph Convolutional Network (GCN) to learn graph-aware patient embeddings [10].
- Multitask Clinical Outcome Prediction and Evaluation: To develop a multitask prediction framework for in-hospital mortality and sepsis classification using separate prediction heads, class-weighted binary cross-entropy and reconstruction losses, and evaluate model performance using AUROC, AUPRC, precision, recall, and F1-score, with particular emphasis on improving recall and reducing false-negative predictions [13, 14].

This project report is structured in following 9 chapters covering different aspects of the project. Chapter 1, introduction provides the overview of the project, where the significance of ICU mortality prediction, existing clinical scoring system and eICU Collaborative Research Database (eICU-CRD) [3] is discussed. Chapter 2, Literature review which covers existing work in the domain of ICU mortality prediction, time series clinical data analysis [15] and sequential neural network architecture. Chapter 3, Problem definition and objectives which defines the problem of research work. This chapter also includes the aim and objectives of the research work. Chapter 4, Proposed methodology and work description where overall project flow like hospital-wise data partitioning, data pre-processing, model training and performance evaluation is described. Chapter 5, Proposed algorithms. In this chapter the neural network architecture used for the prediction of mortality is presented [13]. This chapter also covers principles of the designed models and the pseudocode of the same. Chapter 6, proposed flowchart and block diagram. The system flow and major components are depicted using the flowchart and block diagram. Chapter 7, implementation (Tools and technology used). The tools and libraries and frameworks and technology used in project development are discussed. Chapter 8, result discussion and analysis. In this chapter, experiment results are presented. Comparison of performance of developed models using proper evaluation metrics is presented. Chapter 9, Conclusion and future scope. The major findings of the study are summarized. Future scope and improvements of the study are discussed.

SECTION 2: LITERATURE REVIEW

2.1 Overview

This chapter deals with the approaches to ICU outcome prediction. The main emphasis will be made on the following aspects: eICU Collaborative Research Database [3], temporal clinical data [16], multimodal representation learning [7], patient similarity graphs, Graph Convolutional Networks (GCNs) [10], and hospital mortality/sepsis prediction [17]. Integration of time series data with tabular data is also discussed.

2.2. ICU Mortality Prediction and Clinical Significance

Identification of critically ill patients at higher risk of death is vital in a hospital. The prediction power relies on their performance and on their validation and generalizability [1, 18]. There can be two different mortality outcomes. ICU mortality and hospital mortality. ICU mortality is the death of a patient during the ICU admission period. Hospital mortality is the death of a patient prior to discharge from a hospital, regardless of a ward in which it happens. Consequently, the definition of an outcome should be made clearly when reviewing any related studies.

There were several significant limitations in the development, validation, and reporting of the machine learning models based on routinely collected ICU data identified by Shillan et al. [18] Moreover, according to the review by Sun et al., [2] 40 studies with 317,028 patients and 123 prediction models reported a summary AUROC of about 0.83. However, there was a considerable difference in study methodology and lack of external validation in these results [2] Therefore, it is vital to validate models on data other than the one the model was trained on. Hence, hospital mortality prediction needs clear outcome definition and proper evaluation metrics. Standardized evaluation libraries such as Scikit-learn [19] ensure the reproducible calculation of critical threshold-dependent and independent metrics, while comprehensive metric analyses [14] provide mathematical justification for using F_β and AUPRC over standard accuracy in imbalanced clinical domains

2.3. ICU Datasets and the eICU Collaborative Research Database

The eICU Collaborative Research Database (eICU-CRD) is a multi-center critical-care database which contains information related to more than 200,000 ICU admissions across 208 U.S. hospitals. The potential of eICU-CRD for analyzing clinical outcomes in different healthcare facilities has been described by Pollard et al. [3] earlier. The database contains various types of clinical data, which include laboratory values, vital signs, drugs administration, interventions, diagnoses, and Acute Physiology and Chronic Health Evaluation (APACHE) severity data. These data could be linked to create patient profiles that would unite physiological observations with the clinical context [3]. Other databases like MIMIC-III [20], MIMIC-IV [21], and AmsterdamUMCdb [22] are widely used in critical-care research as well

[5,6,7] However, differences in the population of patients, care settings, and data collection methods prevent comparison between studies.

The multi-center nature of eICU-CRD [3] database makes it especially relevant for studying the generalizability of models across hospitals. Yet, the use of multi-center data is not sufficient for ensuring generalizability on its own; testing must involve making sure that no hospitals from the testing set were included in the training set.

Table 2.1: Comparison of Different Clinical Databases

Dataset	Size	Main Characteristics
eICU-CRD [3]	208 US hospitals; 200,859 ICU stays	Multicenter data, useful for studying differences between hospitals and testing model generalization
MIMIC-III [20]	One US hospital; 53,423 distinct ICU stays	Detailed records from 2001–2012, widely used for developing and evaluating clinical prediction models
MIMIC-IV [21]	One US hospital; 73,181 ICU stays in version 2.2	Updated clinical database covering 2008–2019, with hospital and ICU records
AmsterdamUMCdb [22]	One Dutch medical center; 23,106 ICU and high-dependency unit admissions	Detailed clinical observations, including vital signs, laboratory results, device measurements, and medication records

2.4 Machine Learning for Mortality Prediction

Classic machine learning models such as Logistic Regression [23], Random Forest [24], and XGBoost [25] have been extensively used for ICU outcome prediction. Logistic Regression has a basic level of explainability, while tree-based ensembles can work with non-linear interactions between clinical features. A machine learning model's performance starts to get constant as more data is fed, but a deep learning model's performance increases when more data is made available.

The study conducted by Zhao et al. [17] estimated several machine learning models and neural networks' performance on eICU-CRD dataset in predicting mortality. According to the authors, their analysis demonstrated that predictive power depends on the type of model, target variable, and prediction horizon; thus, comparison across studies should be approached carefully. RNNs are able to process sequential data, and LSTM and GRU networks can effectively learn dependencies between sequences. LSTM encoders can encode the results of lab tests, treatments, and medications.

2.5. Missing Data Clinical Time Series

Missing data is a significant problem in ICU prediction due to non-uniform collection of clinical measurements. Lab tests and other measurements might be conducted with higher frequency when a patient needs close monitoring. Thus, missing data can actually depend on the process and condition of a patient. Missing data is classified into three types [15]: MCAR, MAR, MNAR. Awad et al. [16] worked on the problem of hospital mortality prediction from ICU data and found out the relevance of the observation window size and available clinical features. Che et al. [6] developed GRU-D recurrent neural network model with missing data indicators and information about how long-ago previous measurements have been done, which shows that missing data patterns can contain valuable information for the prediction problem.

The two ways to deal with missing data. Zero-filling replaces zeroes to timesteps where a sequence has missing values. This can make a missing value as an actual recorded observation. The other method is called Forward-filling; it replaces a missing measurement with the most recent recorded value for that patient. Missing data can be both useful and problematic. Che et al. [6] showed that missingness patterns can actually help improve clinical predictions. Their GRU-D model uses both information about which measurements are missing and the time since they were last recorded.

2.6. Deep Learning

Modern clinical deep learning is implemented using software libraries. Data processing ingestion is handled by Pandas [26] and NumPy [27]. The neural architectures are constructed using libraries like PyTorch [28]. Optimizers such as Adam [29] are standard for optimizing convergence when paired with frameworks like Optuna [30] can automate hyperparameter tuning and give optimum hyperparameters that would not be possible to find manually. These methods will be applied to sequential models like LSTM and GRU. Long Short-Term Memory (LSTM) is a type of recurrent neural network designed to learn from sequential data over time. An LSTM model has a feedback mechanism which uses earlier timesteps to predict future results. It is an extension of RNN architecture. Hochreiter and Schmidhuber [4] introduced LSTM to address its limitations. An LSTM consists of three gates and a memory cell. The forget gate decides what information to discard from the hidden state, the input gate adds new important information, and the output gate determines the new hidden state for the current timestep. In healthcare, a condition like sepsis can evolve, so LSTM is useful.

Gated Recurrent Unit (GRU) is a recurrent neural network architecture, introduced by Cho et al. [5]. It learns from previous observations to learn patterns. GRU has a simpler structure with two gates: the update gate which controls how much previous information is stored in the hidden state, and the reset gate which controls how much past information is used in the current timestep. Because GRU has a simpler design compared to LSTM. It can require fewer parameters and may train faster than LSTM. The performance of an LSTM or GRU model depends on the dataset and the prediction task.

Since the eICU-CRD contains two modalities for a patient, one modality alone could not describe the patient's full condition. Multimodal fusion combines embeddings of both modalities into a unified embedding. Wang et al. [7] presented Multimodal Fusion Network (MFNet) for predicting the outcome of ICU patients based on eICU-CRD. This model integrates modality-specific encoders with fusion techniques to extract correlations between different data sources. Graph-based representations are also used in this model. The results presented show that the integration of heterogeneous clinical data is important. One approach that stands out is multi-head attention [31].

Each patient is represented as an individual vector. This approach focuses on a single patient's data ignoring "inter-patient relational topology", which means similar connections between patients. A graph can be created using k-NN [12] and cosine similarity across patients. Each node represents a patient, and each edge represents similarity. Edges in the graph of patients from different hospitals are forced to be zero. This is done to avoid "Practice-pattern Leakage". If the graph connected patients across different hospitals, the model could memorize the specific hospital practices rather than learning the general patterns of the patients. Ma et al. [11] developed a patient similarity graph method for the prediction of ICU mortality with the help of a graph attention network (GAN).

GCNs generate the node representations by fusing information from the node itself and its graph neighborhood. The GCN model widely used by Kipf and Welling [10]:

$$H^{(l+1)} = \sigma\left(\widehat{D}^{-\frac{1}{2}}\hat{A}\widehat{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right) \quad (2.1)$$

In a graph convolution network, multiplying the feature matrix by the adjacency matrix ($\hat{A}$) means a node only aggregates information from its connected neighbors and completely ignores its own features. This is why self-loops are added. If features were aggregated without normalization, the numerical values of high-degree nodes would explode, and low-degree node features would shrink. Normalizing the adjacency matrix by the node degree scales down the data. This stabilizes the variance of feature vectors across multiple layers. In a GCN, there can be problems like oversmoothing [32] are solved using strict depth limits and dropout [33].

Mortality and sepsis prediction tasks are similar and different. The mortality prediction task predicts the survival of the patient until discharge, while the sepsis task tries to predict if the patient belongs to those who are associated with the sepsis. It is crucial to define them distinctly. In this project, the definition of sepsis depends on the ICD-codes and APACHE admission-diagnosis information. Since there is no information about the sepsis onset time, the outcome should be called sepsis identification. The systematic review conducted by Nikravangolsefid et al. [9] on machine learning models for mortality prediction of critically ill adults with sepsis considered 31 studies, which included more than 1.48 million people, revealed huge heterogeneity in methods and a high risk of biases in many cases. They underline the importance of clear definition of an outcome, adequate model validation, and external

validation [9]. There is a relevant study, mortality prediction among sepsis-infected patients in the ICU, done by Wang and Li [8]. They use a multimodal approach including information on the patient and temporal and graph-based modeling. This study proves the relevance of multimodal and graph-based learning using information from different clinical records.

Multitask learning represents one of the ways to train two related tasks using embeddings shared between tasks and task-specific prediction heads [13]. Embeddings sharing could be helpful because both tasks can use the same clinical patterns but make decisions separately. Shared multimodal graph representation is used together with different prediction heads for hospital mortality and sepsis. Class-weighted binary cross entropy losses [34] help to solve problems of class imbalance of the positive and negative class and tabular reconstruction loss is applied to learn static clinical representation.

It is investigated that ICU mortality prediction using classic machine learning methods, temporal deep learning, multimodal fusion, and patient similarity graphs. Papers like MFNet [7] and patient-similarity graph models [11] highlight patient representations and relationships. But performance still depends heavily on the task, the dataset, graph construction, class imbalance, and the chosen model. There are three key factors relevant to this project. temporal clinical observations and static patient features bring different kinds of information. Patient similarity graphs can be used to include relationships beyond what individual feature vectors allow for and validation across hospitals should be considered when evaluating performance of a model facing data from hospitals not seen during training.

SECTION 3: PROBLEM DEFINITION

3.1. Problem Statement

The aim of the project is to predict outcomes based on the data collected from ICU patients including time-series sequential and tabular static features. The task is complicated by missing values, changing state of the patients, and different healthcare settings. The models that are built on data collected at only a few hospitals learn specific patterns which cannot be generalized to other hospitals.

The framework is built for two clinical outcomes: hospital mortality and sepsis classification. The evaluation is performed using hospital-specific training, validation, and test splits.

3.2. Success Criteria

The project will be considered successful if the implemented framework meets the following criteria. The timesteps are compressed into 12×92 tensors. Temporal embeddings generated through LSTM or GRU should achieve a test mortality AUPRC ≥ 0.4, AUROC ≥ 0.8. The temporal embeddings are fused with static tabular embeddings using the best fusion technique. After fusion, the 64-dimensional embeddings should achieve a test mortality AUPRC ≥ 0.4, AUROC ≥ 0.8. The patient similarity graph constructed should contain no cross-hospital edge and should be the sparsest graph possible. A GCN with a joint loss for mortality and sepsis must achieve a test mortality AUPRC ≥ 0.4, AUROC ≥ 0.85 and sepsis AUPRC ≥ 0.6, AUROC ≥ 0.9.

SECTION 4: PROPOSED METHODOLOGY AND WORK DESCRIPTION

The proposed methodology develops a multimodal deep learning framework for hospital mortality prediction and sepsis classification. The architecture integrates time series and tabular features, combines the features and uses a patient similarity graph to find relationships between patients and then predict the outcome.

4.1. Hospital-Wise Data Splitting

The eICU database contains various hospitals data into different files. First, directories for every hospital were created; each directory contains two folders named tabular and timeseries containing the data of the respective modality. 208 Hospitals data was segregated.
When considering data splitting, two approaches presented themselves:

i. **Patient-wise split:** Patients from Hospital A may appear in training, validation and testing. The model can learn patterns associated with Hospital A's patient population and clinical practices before being tested on other patients from that same hospital.
ii. **Hospital-wise split:** Every patient from Hospital A belongs to one partition only. If Hospital A is assigned to training, none of its patients appear in validation or testing.

The main purpose for this hospital-wise splitting was to prevent data leakage. This is a stricter test than randomly splitting patients from the same hospitals across training, testing, and validation. It is to test whether the model can make predictions on hospitals it has not seen during training. 70% of hospitals were used for training, 15% for validation, and 15% for testing. This provides a more realistic evaluation and result.

Table 4.1: Hospital-wise Partition

Partition	Approximate hospitals
Testing	31
Validation	31
Training	146
Total	208

4.2. Timeseries Data Preparation and Preprocessing

Timeseries data preparation converts irregular features into a consistent sequence that a model can process. The prepared timeseries data is processed into a parquet file containing multiple records for each patient stay. The dataset has 74 columns with identifiers and 46

numerical features. Three columns with information were excluded. The target feature is present in the tabular file while all other data is present in timeseries file. To ensure the data is suitable for processing, observations are put in four-hour bins. The bin is calculated using:

$$hour_{bin} = \left\lfloor \frac{t}{240} \right\rfloor, \qquad \text{t: offset in minutes} \qquad (4.1)$$

In the LSTM model, 12 bins were created and in the GRU model, 18 bins were created. Measurements belonging to the same interval are represented in that interval's feature row. Using a fixed window gives the model a consistent time range for learning how a patient's condition changes. Patients with fewer recorded observations have fewer valid timesteps. A Forward-fill is implemented to handle missing values.

4.3. LSTM and GRU Model

LSTM and GRU models were implemented and fine tuned upon several trials based on validation AUROC and AUPRC. The following table lists all the hyperparameters of the models:

Table 4.2: Hyperparameters of Sequential Models

Hyperparameter	LSTM	GRU
Input size per timestep	92	74
hidden size	120	128
Number of dense layers	3	2
Dense units	32	32
Dropout	0.2383	0.3
Learning rate	0.0009	0.001
Optimizer	Adam	Adam
Batch size	64	16
Early stopping patience	5	3

4.3.1. LSTM

After preprocessing each patient stay is represented as a sequence of steps. 12 timesteps for the first 48 hours with 46 numerical features per timestep. 46 additional indicators were concatenated for each feature that indicated whether the feature was an observation or a missing value filled forward. The input shape for one sequence is: $X \in R^{92 \times 12}$. . This means 92 total features for every timestep.

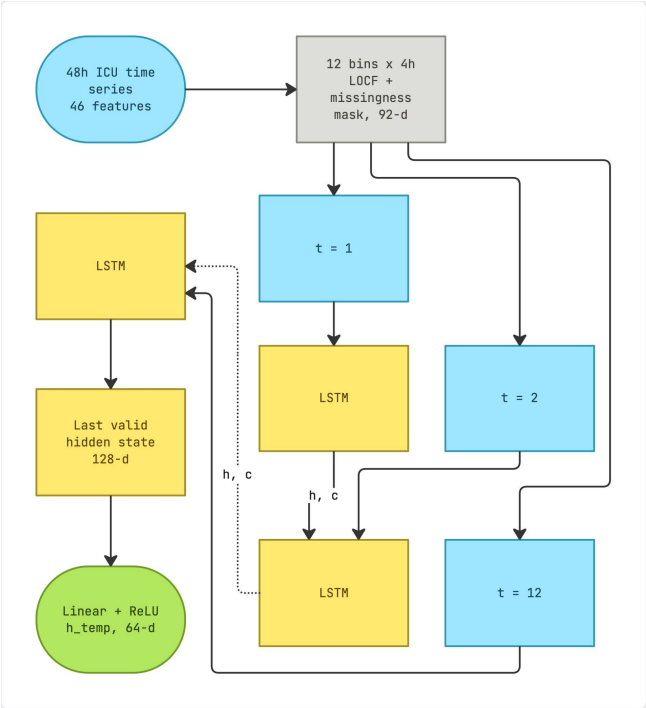


Fig 4.1: Data flow of LSTM Model

The LSTM model was trained using the Adam optimizer. The optimizer updates the trainable parameters during backpropagation. Weighted binary cross-entropy with logits was used as the loss function. The model outputs a logit instead of a probability. This logit will go into an activation function, here, sigmoid and then passed to the BCE loss function. This is done as a single step for numeric stabilization. ReLU function was used to introduce non-linearity. Sigmoid and Tanh functions are used inside the LSTM gates. Gradient clipping was applied to prevent exploding gradient problem. Early stopping was used to stop training when validation performance no longer improved to reduce overfitting. Dropout was implemented and found to be 0.3. The below figure describes the optimization history on validation AUPRC on subsequent trials with different hyperparameters (left). The model was evaluated based on validation AUPRC. fc_layers was found to be the most important hyperparameter followed by dropout and learning rate. On changing those parameters, the AUPRC was found to change significantly.

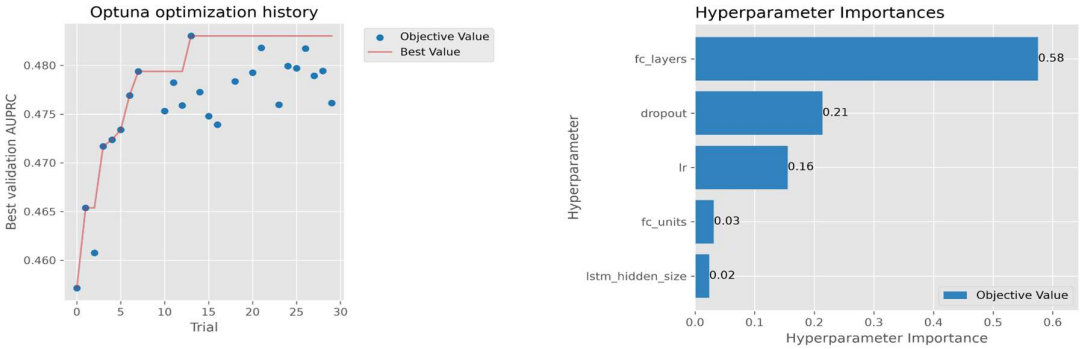


Fig 4.2: Optimization history (left), Importance of hyperparameters (right) of LSTM

4.3.2. GRU

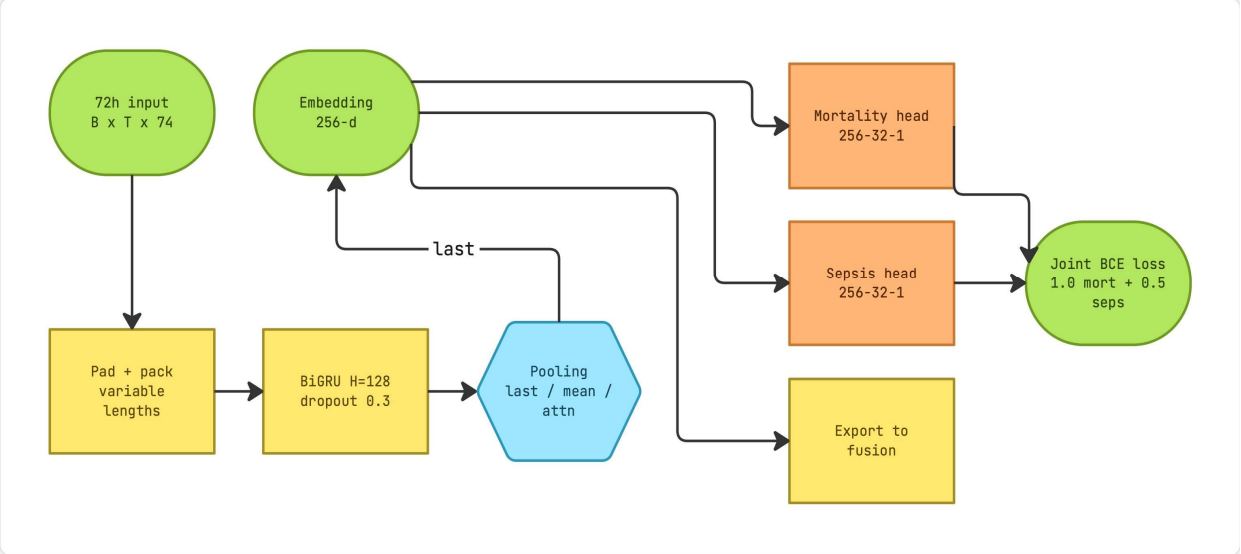


Fig 4.3: GRU Data Flow

The Gated Recurrent Unit was implemented as an alternative approach for temporal embeddings. It was designed to jointly support mortality and sepsis. GRU used the first 72 hours of each patient stay divided into 18 four-hour bins. The input shape for each sequence is $X \in R^{18 \times 74}$

The model was trained for both hospital mortality and sepsis by using a weighted BCE loss. The task loss was combined using weights of 1.0 for mortality and 0.5 for sepsis. The model was trained using Adam optimizer with a learning rate of 0.001. A weight decay of 0.001 was also applied. Early stopping with a patience of 3 was used to prevent overfitting. We use 1

bidirectional layer because it is found that processing the sequence in both directions produces better embeddings. A unidirectional model can only look at past data up to the current time step. A bidirectional GRU uses the complete 72-hour clinical window during training. However, this doubles the dimension of embeddings from 128 to 256. Bidirectional models are excellent at generating embeddings from completed stays but unidirectional models are still useful for real-time data where future data is not known.

The GRU model also uses a temporal pooling mechanism. Temporal pooling compresses multiple hidden states across time into a fixed-dimensional vector for each patient. Three distinct strategies were employed. Last pooling extracts the hidden state from the final valid time step. Mean pooling averages the hidden states across all timesteps to capture the patient's condition over the observation sequence. Attention pooling applies to a single layer to assign an attention score to each timestep with Softmax function to introduce non-linearity. Attention pooling is the most suitable among all, but last pooling is also a suitable approach from real-time prediction.

4.4. Multimodal Fusion

The proposed methodology employs attention-based multimodal fusion. The temporal embedding and static embeddings are provided and combined into a single 64-dimensional patient embedding. Multi-head attention is used for fusion. Different fusion methods are implemented and compared. Concatenation fusion concatenates both static and temporal embeddings into 128-dimensional fused embeddings. Additive fusion adds both the embeddings. Gated fusion learns a gate to learn how much information comes from each embedding. All four fusion techniques were implemented, and the output was fed to a Graph Neural Network to find the best fusion technique.

Attention fusion finds the context between the different modalities. Both embeddings are converted into three vectors: Query, Key, and Value vectors. These vectors are found using transformation through trainable weight matrices. The final embeddings generated are context aware of both embeddings. There are 3 types of attention mechanisms: Bahdanau attention, Luong attention, and self-attention. We use a self-attention mechanism with both modalities as each token, so there is a total of two tokens. There is a total of 4 attention heads used, and each head operates 16 dimensions per token. Residual connections are also used as per the self-attention architecture, and layer normalization was also used to normalize the features. A feed forward neural network with ReLU activation and dropout 0.25 followed by another residual connection and layer normalization. Finally, the fused embeddings found are found by mean pooling across the two tokens. The result is 64-dimensional embedding.

4.5. Patient Similarity Graph

Each patient's embedding is then used to construct a patient similarity graph. Each node represents a patient, and each edge represents clinical similarity. Cosine Similarity is used with the formula:

$$S_c(A, B) = \frac{A.B}{|A||B|}. \quad (4.2)$$

A k-NN identifies the k most clinically similar patients. The value of k is a hyperparameter that will be found using multiple trials and then feeding the graph to a GCN and selected using validation AUROC and AUPRC.

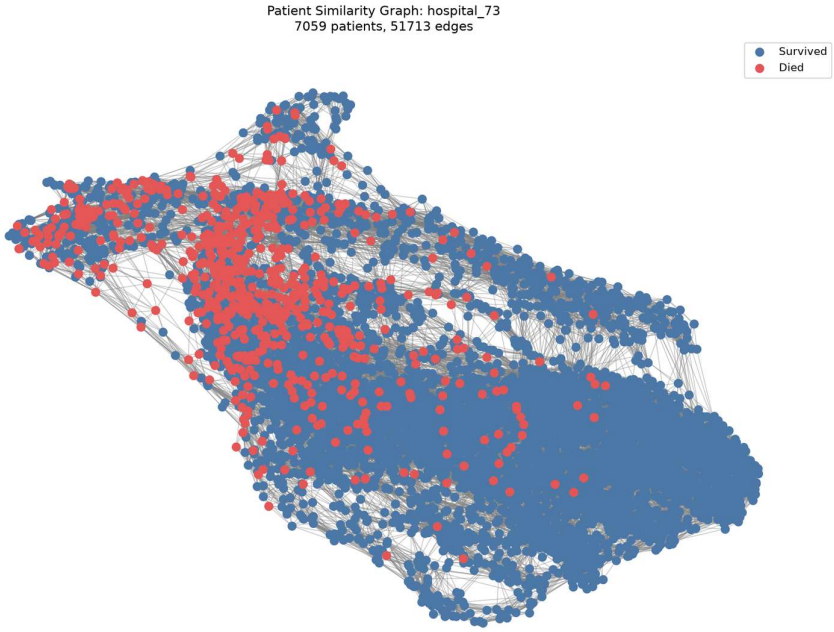


Fig 4.4: Patient Similarity Graph using k = 10

.The image visualizes the intra-hospital Patient Similarity Graph generated for hospital_73 with k = 10. The graph contains 7,059 nodes connected by 51,713 edges. The visible clustering validates the architecture's core premise: patients with high mortality risk share measurable clinical similarities. The GCN uses these shared neighborhood features to identify patients who might otherwise look like outliers. In mortality, similar patients may share combinations of features associated with death and the GCN can aggregate information from neighbors to smooth out the embeddings. A patient similarity graph is very resourceful in sepsis prediction. Since sepsis can evolve significantly in the first admission hours and patients having sepsis will show only a particular set of symptoms, so a GCN can be really powerful in a disease classification like sepsis.

4.6. Graph Convolution Network

We implemented a two-level sparse Graph Convolutional Network (GCN) to generate graph-aware patient representations. The main difference between this model and one that considers each patient individually is that this one aggregate information from other connected patients in the graph. Each node in the graph adjacency matrix is made up of self-loops such that each patient maintains their own information and receives information from other connected

patients. The normalized adjacency matrix is then used to facilitate feature aggregation as discussed earlier. Aggregation of the transformed node features is done at each convolutional layer based on the normalized adjacency list. The two-layered GCN gradually adds information from the neighborhoods to generate graph-aware patient embeddings. The embeddings are then utilized by the prediction heads. The GCN is chosen as the architecture suitable for the project's sparse and homogeneous graph of patient similarity. Other choices included GraphSage and GAN but a GCN with multiple convolution layers was more intuitve than the other choices.

4.6.1. Loss Function

The joint loss function used in GCN:

$$L_{total} = L_{BCE}(\hat{y}_{mort}, y_{mort}; w_{mort}) + \lambda_{seps} L_{BCE}(\hat{y}_{seps}, y_{seps}; w_{seps}) + \lambda_{recon} \parallel \hat{x}_{tab} - x_{tab} \parallel^2 \quad (4.3)$$

- w_{mort}**:** Ratio of negative to positive mortality cases.
- w_{seps}**:** Ratio of negative to positive sepsis cases.
- λ_{seps}**:** Sepsis Loss Multiplier. It is used to weight BCE loss from the sepsis prediction head in the total loss function.
- λ_{recon}**:** Reconstruction Loss Multiplier. It is used to scale MSE reconstruction loss. This acts as a regularizer during training.
- x_{tab}: This represents the original input dimension of the static clinical records. It is a 67-dimensional vector.
- $\hat{x}_{tab}$: This represents the reconstruction of static tabular features. It is a 67-dimensional output generated by the MLP's decoder. The MSE reconstruction loss specifically calculates the squared difference between $\hat{x}_{tab}$ and x_{tab}

The loss function uses Binary cross-entropy to evaluate the hospital mortality prediction ($\hat{y}_{mort}$) against the true value (y_{mort}). w_{mort} is used to counter the class imbalance approximately 10.35. The sepsis loss also utilizes BCE to evaluate $\hat{y}_{seps}$ against y_{seps}. w_{seps} is applied for the class imbalance in sepsis approximately equal to 5.04. The final term of the function calculates a Mean Squared Error between the original static 67-dimensional tabular inputs x_{tab} and the MLP decoder's reconstructed output $\hat{x}_{tab}$ scaled down by a multiplier of λ_{recon}= 0.1, this acts as a regularizer during joint training to prevent latent representation collapse. Latent representation collapse occurs when a model maps different inputs to nearly identical embeddings vectors. This makes the model lose useful distinctions of the representations. A GCN uses aggregation of nodes to update the embeddings, if repeated aggregation makes patient embeddings similar then the model will struggle to distinguish patients with different outcomes. This is a closely related phenomenon to oversmoothing in GCNs. This will affect the recall which is a very important metric in medical.

Why do we not add a time series reconstruction loss? For time series data only, the Temporal LSTM Encoder is used without any decoder. Rather than trying to reconstruct the original 48

hour-long sequence, the LSTM uses sequence masking to pick out only the "last valid" hidden state ($h_{last} = h_{T-1}$). The last state is directly transformed into a dense 64-dimensional embedding (h_{temp}). As there is no mechanism for sequence-to-sequence decoding, there is no temporal output produced by the model to be compared with the input.

4.6.2. GCN Architecture

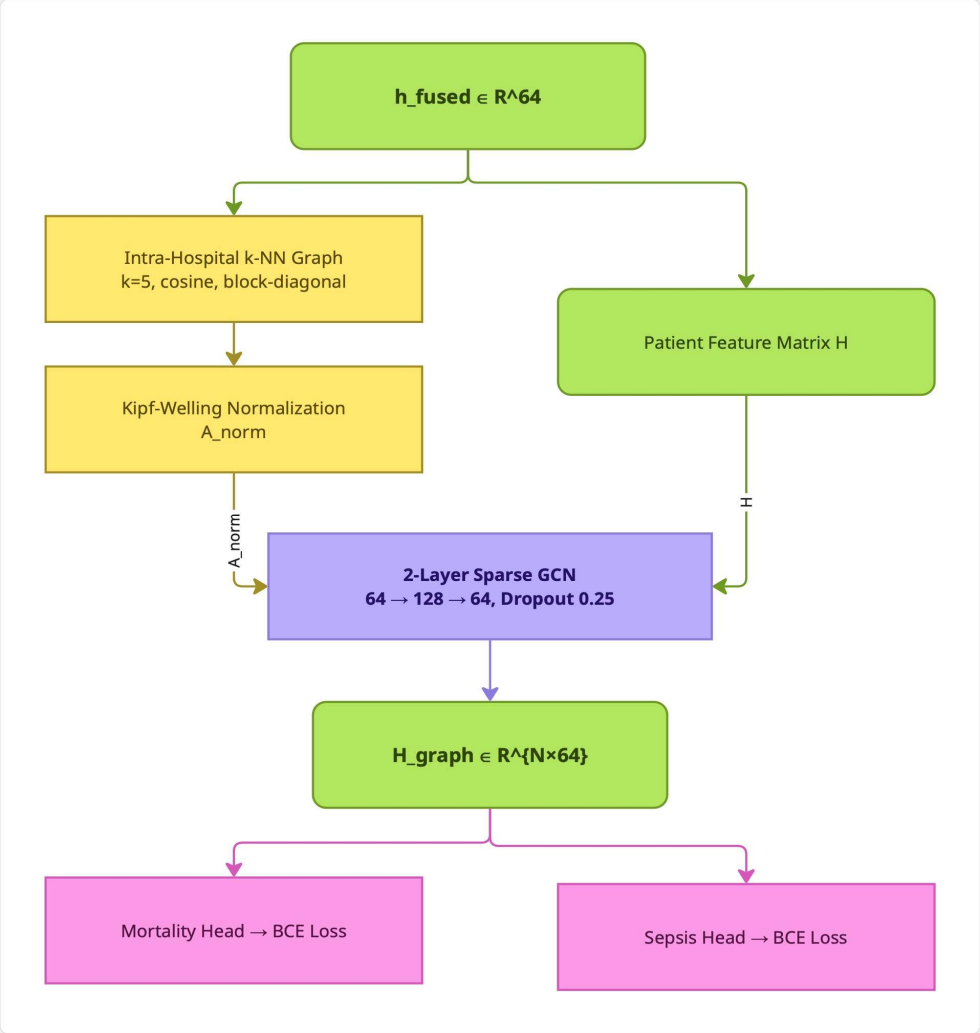


Fig 4.5: GCN Architecture Overview

The pipeline begins with h_{fused} representing the initial node features as an $N \times 64$ matrix. An adjacency matrix is constructed using cosine k-Nearest Neighbor search with k = 5. A self-loop is added to every node, and the features are normalized using the normalization formula. There are two convolution layers in architecture. The first layer ingests the node features alongside the adjacency matrix A_{norm}. It projects the 64-dimensional input up to a 128-dimensional hidden state. ReLU activation function and a dropout rate of 0.25 is applied. The second layer takes the 128-dimensional output from the first layer and A_{norm} reduces the dimensionality

back to 64 using a ReLU activation. The final generated graph embedding H_{gr} , is an N×64 matrix. This structured representation is mapped directly into two separate prediction pathways: a mortality head and a sepsis head.

4.6.3. Mortality Prediction

The architecture for both mortality and sepsis prediction models is the same and consists of 2-layer MLPs that transform the 64-dimensional graph embeddings into 32 dimensions and then to a single dimension using a ReLU activation function (64→32→1). The model calculates mortality head and sepsis head with the equations as below and after generating them, the system evaluates them against the true labels $y_{mort}$and y_{seps}

Mortality Head: This module calculates the raw mortality prediction score $\hat{y}_{mort}$ using the formula:

$$\hat{y}_{mort} = W_{m2}ReLU\left(W_{m1}H_{graph} + b_{m1}\right) + b_{m2} \in R \quad (4.4)$$

Sepsis Head: Operating in parallel, it calculates the raw sepsis prediction score $\hat{y}$ using dedicated learnable weights:

4.6.4. Oversmoothing

Oversmoothing is a problem in Graph Convolutional Networks (GCNs) in which node feature embeddings become increasingly similar and indistinguishable as information is propagated through multiple layers. This occurs because each GCN layer updates a node's representation by aggregating and averaging the features of its connected neighbours. In the initial layers, a node incorporates information from its immediate neighbours, while deeper layers allow information to propagate from neighbours of neighbours, progressively expanding the node's receptive field. As the network depth increases, nodes may eventually aggregate information from similar sets of nodes across the graph, causing their representations to converge and reducing the model's ability to distinguish between different nodes.

Before the GCN, the fused patient embeddings had a similarity of 0.8239. After going through the first GCN layer, the nodes borrow the features of their neighbors and hence increase the similarity to 0.9535. In the second layer, the similarity increases to an extremely high level of 0.9900. By the second layer, the similarity reaches 0.9900. This shows that the patient node embeddings are becoming almost identical, demonstrating oversmoothing.

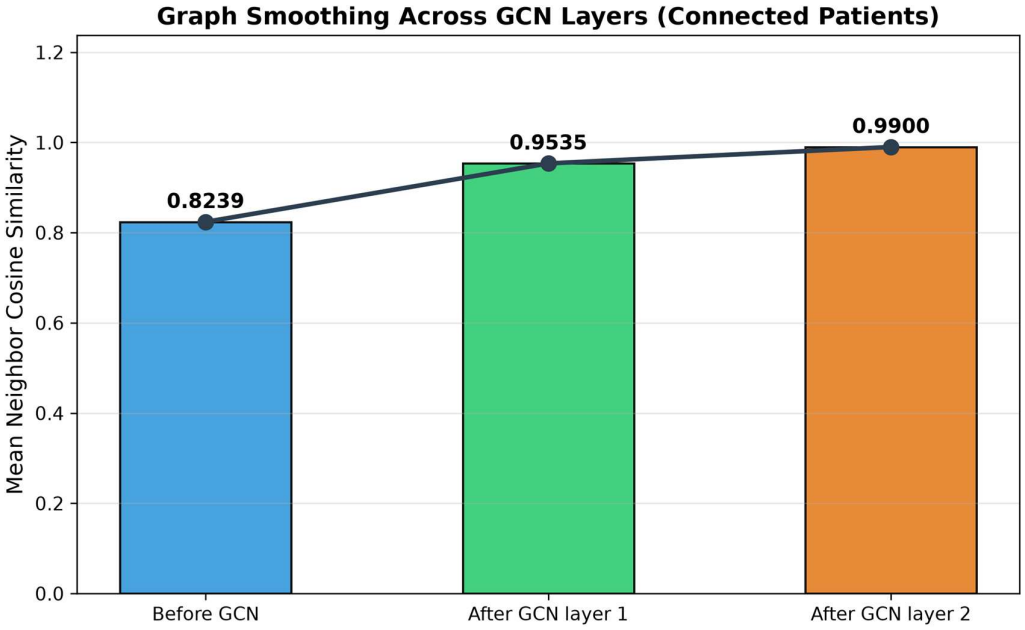


Fig 4.6: Oversmoothing graph of GCN

SECTION 5: PROPOSED ALGORITHMS

5.1. Multimodal Graph-Based Clinical Outcome Prediction Algorithm

The designed model employs temporal clinical information, static patient characteristics, attention-based multimodal fusion, and graph-based learning to predict hospital mortality and sepsis outcomes. The design model is built using the LSTM temporal encoder, MLP for static features, attention fusion layer, patient similarity graph with k-nearest neighbors (k-NN), and two-layer Graph Convolutional Network (GCN).

5.2. Algorithm Pseudocode

Input: D_{temp}: Temporal dataset, D_{tab}: static dataset, Y_{mort}: mortality labels, Y_{seps}: sepsis labels, H : hospital identifiers.

Output: M : Trained model, P_{mort}, P_{seps} : predicted outcome probabilities, E : evaluation metrics.

I. Load temporal records, static patient features, outcome labels, and hospital ids.
II. Partition hospitals into train, validation, and test sets with the constraint of having one hospital in one partition.
III. Temporal preprocessing:
    i. Select records for the first 48 hours of each ICU visit.
    ii. Create 12 slices of observations per 4 hours.
    iii. Fill in missing measurements via pre-implemented imputation techniques and create masks.
    iv. Compute valid-timestep masks and perform normalization of the numerical features based on pre-trained preprocessing parameters.
IV. Static preprocessing: Perform preprocessing pipeline of 67 static features.
V. Initiate an LSTM-based temporal encoder, static feature MLP with decoder, attention-based fusion block, two-layered sparse GCN and mortality and sepsis predictions blocks.
VI. In each training epoch:
    i. Encode each temporal record via LSTM encoder and obtain 64-dimensional embeddings.
    ii. Encode static features via an MLP encoder-decoder and obtain 64-dimensional static embedding.
    iii. Fuse them via attention-based multimodal fusion into 64-dimensional patient embedding.
    iv. Construct sparse k-NN patient similarity graph based on the computed patient embeddings via cosine similarity function with the following constraints: (k=5) and patients can be only connected within their hospitals.

v. Obtain patient representations via two-layered sparse GCN.
vi. Obtain logits for mortality and sepsis via the corresponding blocks.
vii. Compute the joint loss
viii. Backpropagate gradients and update model parameters via optimizer of choice.
ix. Evaluate validation performance and save the best checkpoint by validation criteria.
x. Perform early stopping if the stopping condition criterion is met.

VII. Restore the best checkpoint and obtain predicted mortality and sepsis scores for the held-out test dataset.

VIII. Obtain AUROC, AUPRC, precision, recall, F1-score, accuracy, balanced accuracy and confusion matrices for each prediction task, if available.

IX. Return the trained model, predicted scores, and evaluation metrics.

SECTION 6: PROPOSED FLOWCHART / BLOCK DIAGRAM / DFD

6.1. Overall Flowchart

The flowchart illustrates the complete workflow, from hospital-wise data partitioning to mortality and sepsis prediction.

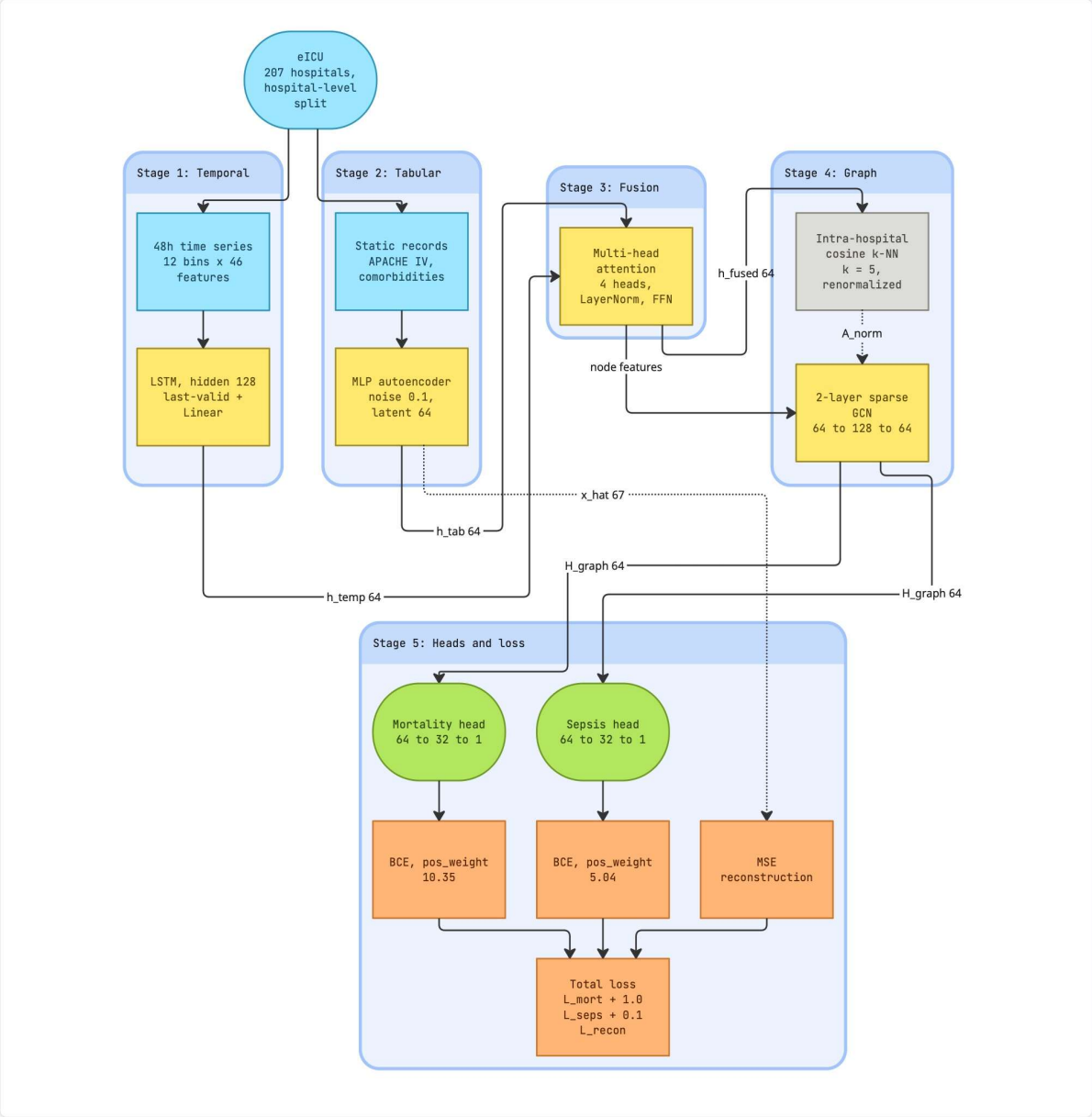


Fig 6.1: Complete workflow

6.2. Hospital-Wise Split Diagram

This flowchart shows how the eICU data set was partitioned into training, validation, and test sets at the hospital level, guaranteeing that each hospital does not appear in more than one partition.

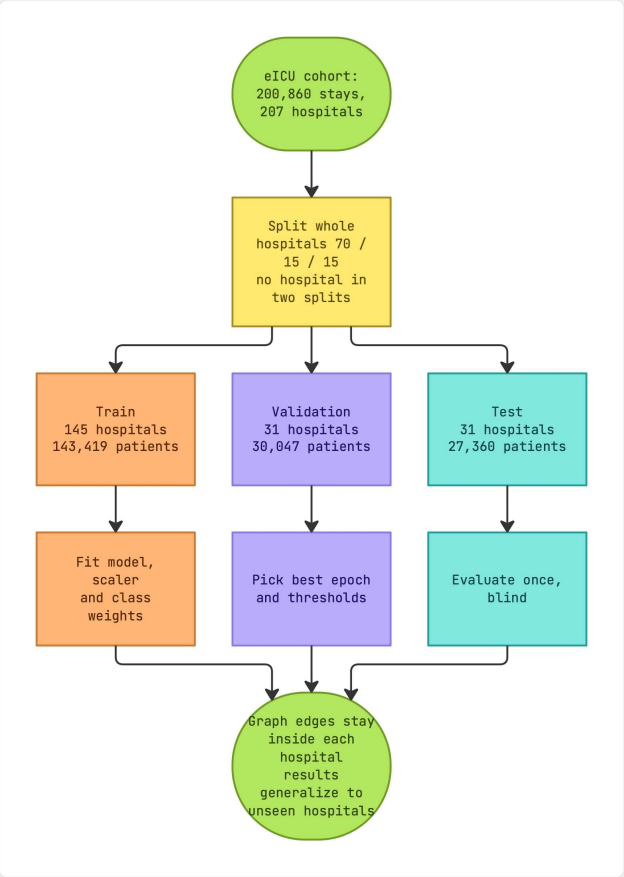


Fig 6.2: Hospital- Wise Split Flow

6.3. Patient Similarity Graph

This flowchart shows how 64-dimensional fused representations of the patient are used to form a sparsity graph. Cosine similarity between the patients from the same hospital is computed, and the k nearest neighbors are selected based on the specified value of k. Afterward, adjacency matrix is made symmetric, self-loops are formed, and matrix is normalized according to Kipf–Welling's method. This leads to the block-diagonal adjacency matrix, which keeps hospitals separate and is then fed to GCN.

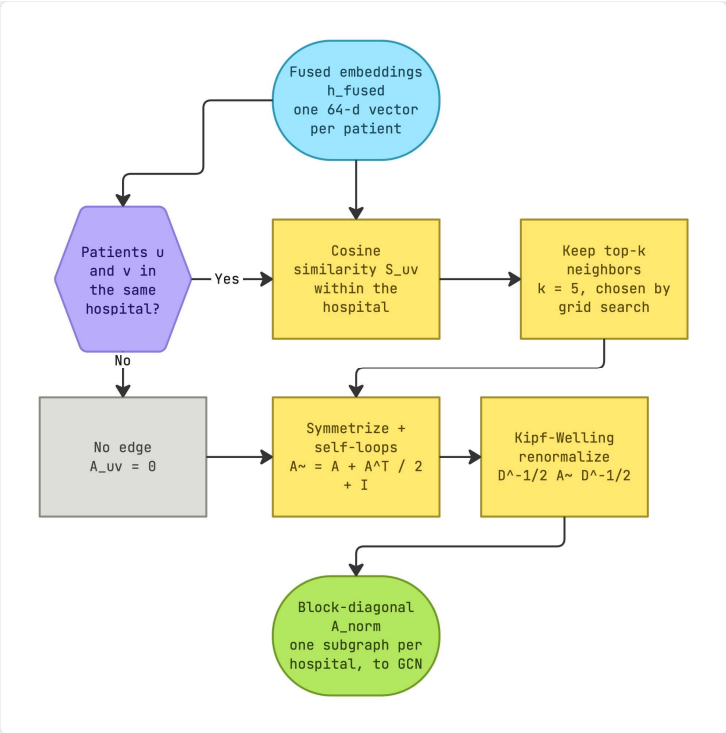


Fig 6.3: Flowchart of Patient Similarity Graph Construction

SECTION 7: IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)

The project was implemented using Python and its libraries. Pytorch was used to build and train neural networks. Pandas, NumPy, and Matplotlib were also used to execute required tasks. Parquet file format was used for storing the prepared tabular and time-series data sets. Scikit-learn was used for partitioning the dataset based on hospitals and evaluation metrics, while Optuna was used for hyperparameters optimization.

7.1. Performance Metrics

The performance of the different models was evaluated using AUROC, AUPRC, Precision, Recall, and F1-score. Area Under the Receiver Operating Characteristic Curve (AUROC) measures the model's ability to distinguish between patients who survive and those who die, with higher values indicating better discrimination. Area Under the Precision-Recall Curve (AUPRC) evaluates the trade-off between precision and recall across different classification thresholds and is particularly useful for the eICU database because mortality events are less frequent than survival events. Precision measures the proportion of patients predicted to die who actually died, with higher values indicating fewer false-positive predictions. Recall measures the proportion of patients who died and were correctly identified by the model, with higher values indicating fewer missed deaths. Finally, the F1-score is the harmonic mean of precision and recall, providing a balanced measure of both metrics.

Accuracy and balanced accuracy were calculated but not used as a performance matrix due to the imbalanced nature of dataset. After training, the selected model was evaluated on the test set. The logits were converted into probabilities using sigmoid function. AUROC and AUPRC were calculated using predicted probabilities. The evaluation was performed separately from model training so that test results did not influence hyperparameter selection. This served as an evaluation of the model's accuracy on the hospitals that were not used in training.

7.2. Dataset Loading

Loading of data set was done through DataLoader. This module was used for loading the data set into the model in batch and during the training phase of the data set. No hand picking of samples was required in DataLoader. It goes through the entire dataset and extracts patient data in batches. Note that the use of DataLoader does not necessarily imply that the entire dataset would be kept outside of memory. This will depend entirely on how the underlying dataset is created. In our project, batching ensures that patients' sequences are loaded into the LSTM in batches while training.

7.3. Hyperparameter Optimization

Optuna was used to explore different LSTM configurations over 30 trials. The hyperparameter importance plot indicates that the number of dense layers had the greatest estimated influence

on trial performance followed by dropout and learning rate. These results helped guide the selection of the model configuration.

During hyperparameter tuning, neurons from 32 to 120 were considered with singular and multiple hidden layers. 120 neurons were found to perform well with 1 to 3 hidden layers as well as 88 neurons with 3 hidden layers performed only slightly worse. The 120-neuron architecture was tested across different activation functions in dense layers. ReLu function was used for dense layers. The LSTM layer used tanh for cell states and sigmoid for output whereas the dense layer performed best on ReLu function. ReLu variants like leaky ReLu and exponential ReLu along with tanh on dense layers performed worse than ReLu.

7.4. Hospital-Wise Splitting and Preprocessing

Hospital-wise data split was implemented using Python, and scikit-learn's train_test_split function. The code identified hospital directories containing both the prepared tabular and timeseries parquet files. Hospitals that missed any of the files were excluded.

A fixed random seed 42 was used to train the model. The hospitals were divided into training, validation, and testing sets using a 70:15:15 split. The hospital partitions were then used to create patient datasets.

The timeseries data was preprocessed using Pandas and NumPy. Since the three list-types of columns served no numerical purpose or pattern for the model, the columns were dropped. That meant 46 numerical features per timestep. Missing data was implemented and compared with both forward fill and zero fill.

Two missing data handling approaches were evaluated for hospital mortality prediction. Forward-fill imputation and zero-fill imputation. Below is the comparison made:

Table 7.1: Comparison of Timeseries Imputation Techniques

Evaluation metric	Forward-fill model	Zero-fill model
Mean test AUROC	0.8446 ± 0.0044	0.8436 ± 0.0052
Mean test AUPRC	0.4643 ± 0.0048	0.4633 ± 0.0072

The forward-fill model achieved a slightly higher mean AUROC and AUPRC than the zero-fill model. However, the differences were small: 0.0010 for AUROC and 0.0010 for AUPRC. Both approaches therefore produced almost similar predictive performance in these experiments.

7.5. LSTM Model

The LSTM classifier was implemented in PyTorch by using a pre-existing class called torch.nn.Module. Each patient sequence contained up to 12 timesteps, with 46 numerical

features and 46 indicators of forward-fill per timestep. The input tensor therefore had the shape (batch size, 12, 92).

A timestep mask was used to identify valid intervals within each patient's sequence. The mask was summed across the timestep dimension to estimate the sequence length, and one was subtracted to obtain the index of the last valid timestep. The output generated by the LSTM layer was chosen for each patient and fed to the model to generate a mortality logit.

The mask indicates the timesteps that are valid instead of indicating the missing values of features. This helps the classifier to pick the output related to the last valid interval instead of choosing the output at the end of the padded sequence automatically.

7.6. GRU Model

The GRU model was implemented using PyTorch. The model takes the first 72 hours of ICU time series data into 18 uniform 4-hour bins. Each timestep inputs 74 canonical features utilizing forward-fill and zero-fill imputation for missing values. Variable length sequences are managed using PyTorch's pack_padded_sequence, pad_packed_sequence, torch.nn.utils.rnn.pad_sequence so that padded positions do not contribute to gradient updates.

The network was trained using the **Adam** optimizer, paired with a ReduceLROnPlateau learning rate scheduler. ReduceLROnPlateau is a dynamic learning rate scheduling algorithm provided by PyTorch. It monitors a specific evaluation metric (such as validation AUROC or validation loss) during training and automatically reduces the learning rate when that metric stops improving. When no improvement is seen on validation data, after 2 epochs the current learning rate is multiplied by 0.5 to help the model converge

7.7. Attention Based Multimodal Fusion

The project implements a closely related Transformer architecture. It was implemented in PyTorch using nn.MultiheadAttention, nn.LayerNorm, and a feed-forward neural network. Temporal and static embeddings are stacked. A multi head attention with 4 heads was implemented. It allows relationships between modalities. Layer normalization was used to reduce overfitting.

7.8. Graph Convolution Network

The GCN implementation was carried out using two modules: PatientGraphGNN, a dense Graph Convolutional Network module used during batch training, and torch.sparse.mm, which enabled efficient sparse matrix multiplication to process large graphs across the full data split while reducing memory consumption and preventing memory overflow errors.

7.9. Overfitting and Optimization Techniques

Overfitting occurs when a model learns patterns that only exist in the training data. Overfitting can lead to misleading performance where training performance is much better than actual

performance on test data. In an ICU healthcare setting, this can result in missing deaths or incorrect mortality prediction.

We used validation performance to identify overfitting. While training loss generally decreased, validation performance fluctuated and stopped improving. This led to early stopping of training as the model will only overfit over the remaining epochs.

Some of the methods used to improve training and minimize overfitting include dropout, early stopping, optimizer, and gradient clipping. Dropouts of varying degrees beginning from 0.1 were employed in different ways. Early stopping with varying patience was implemented. There is no built-in class or method for early stopping, so it was implemented from scratch. Adam optimizer combines momentum and adaptive learning rates for faster convergence. It was implemented using torch.optim.Adam with a learning rate and weight decay. During training, loss.backward function calculates gradients and optimizer.step function applies the parameter updates. Gradient clipping was also applied to prevent very big values from occurring. It happens after backpropagation and before Adam optimizer. This prevents exploding gradient problem. This was implemented using clip_grad_norm function which calculates the global norm of the gradients, if norm exceeds max_norm (max_norm = 1.0), PyTorch scales the gradients down proportionally.

SECTION 8: RESULT DISCUSSION AND ANALYSIS

8.1. Training Behavior

Convergence of the model and overfitting issues were evaluated using the training and validation plots. In general, there was a decrease in training loss for all epochs, which is a sign of the learning process by the model. At the same time, there were oscillations in the validation loss along with increasing it for the last epochs, whereas validation AUPRC remained almost constant. It can be seen that the model was overfitting at some point.

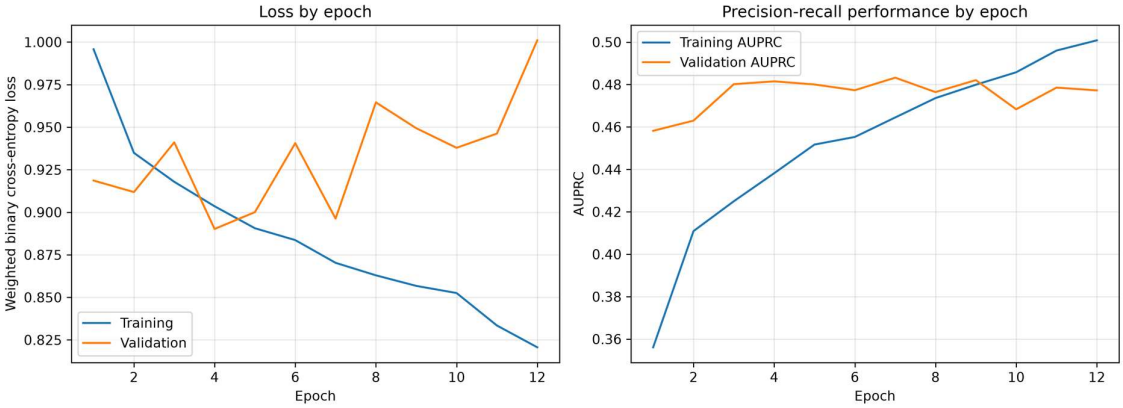


Fig 8.1: Loss by Epoch (left), AUPRC per epoch (right)

8.2. Discrimination Behavior

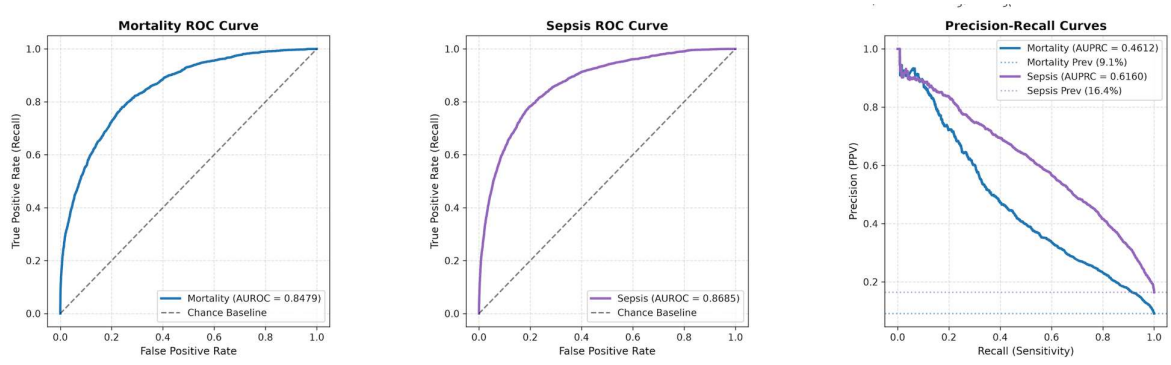


Fig 8.2: Classification Curves for LSTM

The ROC curve was used was used to assess the model's ability to distinguish between patients who survived and those who died before hospital discharge. The area under the ROC curve (AUROC) summarizes discrimination across classification thresholds. Mortality AUROC was

8.3. Classification Performance of Time Series Models

8.3.1. LSTM Model

The confusion matrix was used to determine the number of true positives, true negatives, false positives, and false negatives at a probability threshold of 0.72 for mortality and 0.42 for sepsis. This will provide more insight into the types of classification errors made by the model.

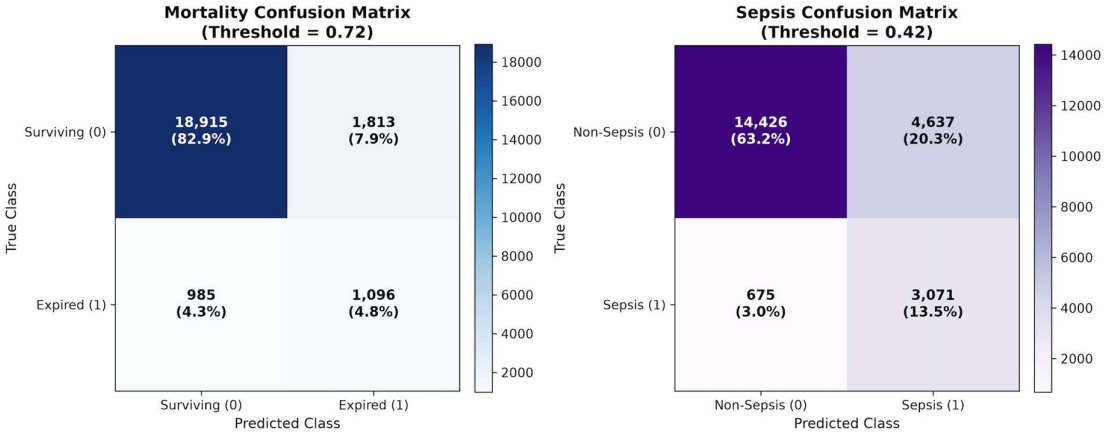


Fig 8.3: Confusion Matrix of Mortality and Sepsis of LSTM

It is to be noted that in both confusion matrices, false negatives are the lowest. In a clinical setting, focus is to the maximize the number of patients predicted alive but died. Recall is the metric that indicates false negatives.

The performance metrics was found to be:

Table 8.1: AUROC and AUPRC of LSTM Model

Metric	Mortality	Sepsis
AUROC	0.8491	0.8684
AUPRC	0.4680	0.6159

The LSTM model was tested across different seeds and trained to perform consistent results. This means that the model is able to generalize patient results. The three random seeds: 42, 123, and 999. The AUROC scores remained relatively stable, ranging from approximately 0.838 to 0.848.

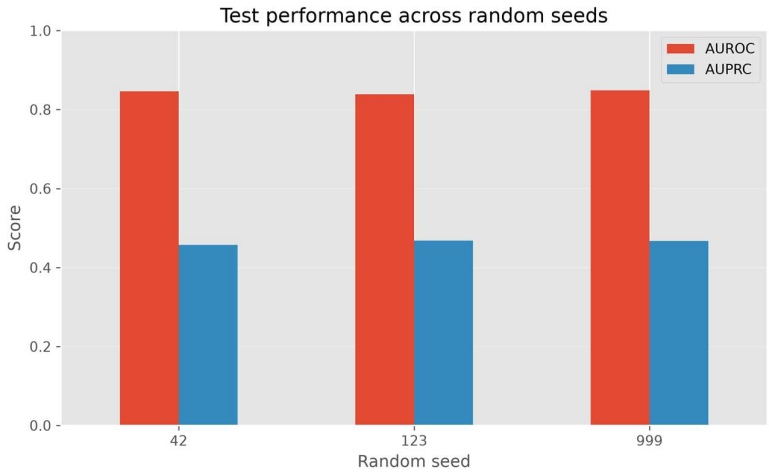


Fig 8.4: Test Performance across seeds 42, 123, 999

8.3.2. GRU Model

A search involving 20 hyperparameter trials was conducted to find the best configuration. The search changed hidden dimensions, recurrent layers, dropout, temporal pooling strategies, learning rates, batch sizes, weight decay, and task-loss weights. All 20 trials completed successfully, with validation mean AUROC values ranging from 0.8241 to 0.84

Table 8.2: AUROC and AUPRC of GRU Model

Metric	Mortality	Sepsis
AUROC	0.8481	0.8408
AUPRC	0.4680	0.5881

The hyperparameter search provided insight into the configurations of different GRU models. 128 hidden units performed consistently well. Dropouts of 0.2 and 0.3 performed well and were common in the best configurations. Bidirectional GRUs appeared in four of the five highest-ranked configurations. A bidirectional GRU can capture contextual information from across the observation window. This can be useful when learning retrospective patient representations. The last valid timestep pooling strategy also performed consistently well. It provides a representation of the patient's trajectory without requiring implementing a separate attention mechanism.

8.3.3. LSTM and GRU Comparison

Both LSTM and GRU models were compared for the best temporal embeddings. The results show that both architectures achieved useful discrimination. LSTM's mortality AUROC was marginally higher than GRU's mortality AUROC but the difference is almost negligible.

Table 8.3: Comparison of LSTM and GRU

Criterion	LSTM	GRU
Temporal architecture	Cell state with input, forget, and output gates	Update and reset gates
Reported mortality AUROC	0.8491	0.8481
Reported mortality AUPRC	0.4680	0.4464
Sepsis validation AUROC	0.8684	0.8408
Sepsis validation AUPRC	0.6159	0.5881
Role in final project	Selected temporal encoder	Alternative temporal experiment

8.4. Multimodal Fusion

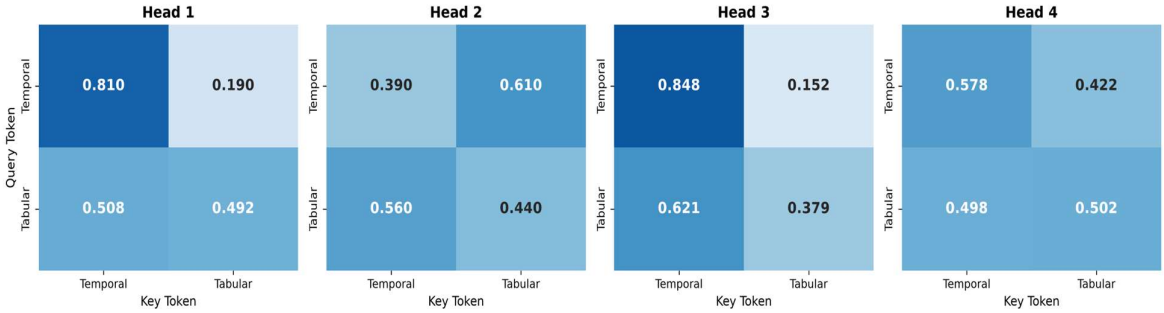


Fig 8.5: Attention Based Fusion Heatmap

Temporal queries are allocated 0.810 attention from Head 1 in terms of temporal keys. Head 2, on the other hand, allocates 0.610 attention to tabular keys. Head 3 is strongly biased towards the temporal modality with 0.848. Attention is equally split in Head 4. Across the four heads, the average of the temporal query key allocation is 0.656 to temporal keys and 0.344 to tabular keys.

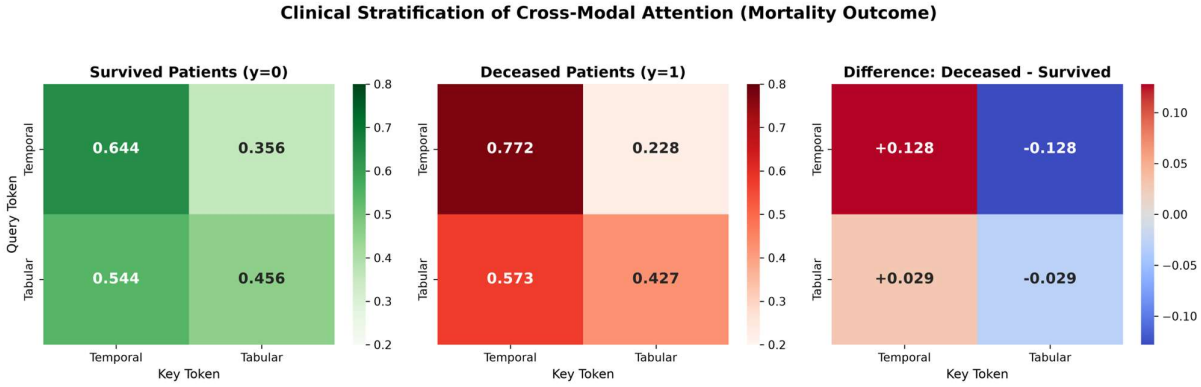


Fig 8.6: Stratification of Attention

In the case of survived patients, the mean value of temporal-query-to-temporal-key attention weight equals 0.644, while the value for deceased patients equals 0.772. The difference of 0.128 means that there is a higher average attention allocation between temporal queries and temporal keys in the deceased group. The attention to temporal keys increases from 0.544 among survived patients to 0.573 among deceased patients.

It seems that the model's attention allocation between temporal and static representation depends on the mortality outcome observed, especially in case of temporal queries. The four attention mechanisms results is as follows:

Table 8.4: AUROC, AUPRC and Accuracy of Different Fusion Techniques

Metric	**Concatenation**	**Attention**	**Gated**	**Additive**
Threshold	0.7830	0.7592	0.7666	0.7434
AUROC	0.8782	0.8793	0.8788	0.8767
AUPRC	0.5330	0.5292	0.5289	0.5237
Accuracy	0.9033	0.9022	0.9046	0.9021
Precision	0.4733	0.4692	0.4794	0.4680
Recall	0.5295	0.5451	0.5257	0.5276
F-1 score	0.4998	0.5044	0.5104	0.4960

Therefore, Attention based fusion gives the best recall with an increase of 0.2 among all. Recall is prioritized to result in fewer false negatives. Also, it had the highest AUROC and 2nd best F1-Score among all, so multi-head attention was used for fusion.

8.5. Patient Similarity Graph

The appropriate value of k was found through testing of different values of k. The best value of k will result in a sparser graph with strong similar connections. A fewer k captures highly local similarities while a high k captures noise and can result in oversmoothing of the graph when giving it an input to any Graph-based neural network. So, these were the results of GCN for different values of k:

Table 8.5: Mortality and Sepsis AUPRC of GCN for Different Values of k

Neighbors (k)	Mortality AUPRC	Sepsis AUPRC
1	0.524	0.371
3	0.520	0.370
5	0.520	0.369
10	0.519	0.369
15	0.519	0.368

k = 1 provides the highest AUPRC. These results do not show that k = 5 is the best choice. Since the performance was stable among different values of k and the differences almost negligible, k = 5 was chosen. A small k such as 1 can miss useful context from other relevant patients with similar conditions. We want a balanced neighborhood with minimum noise and useful similarities that can connect to as many patients without connecting too many dissimilar patients. A big k like 10 was not used because then there is a chance it might not perform well on unseen data.

8.6. Graph Convolution Networks

The GCN network showed promising results well above the success criteria. The GCN network was tuned using validation data and evaluated the test data to measure its final metrics. The results show valid learning as the loss decreases in subsequent epochs.

8.6.1. Performance of GCN

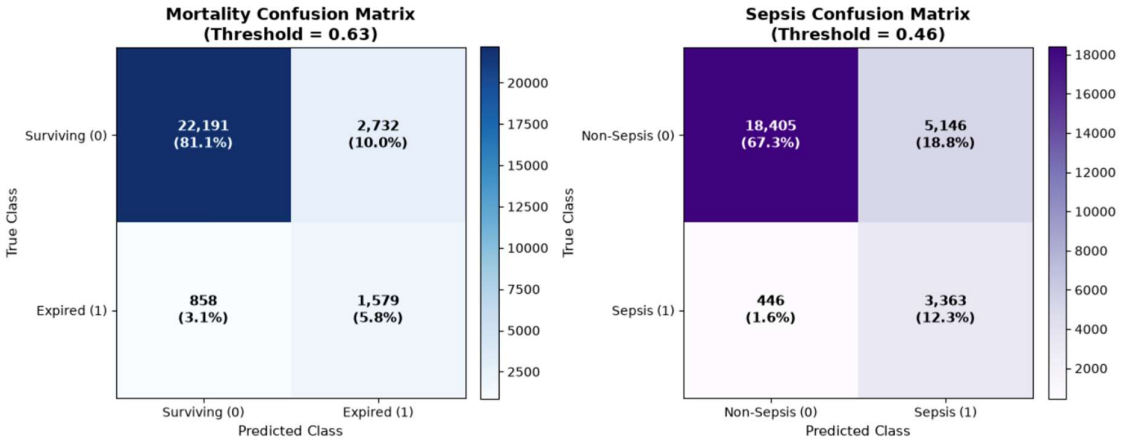


Fig 8.7: Confusion Matrices of GCN

The above figure shows that the model correctly classifies a large number of patients who lived. Least number of misclassifications are done where the patient died but the model predicted life. The graph below depicts ROC curve and PR curves.

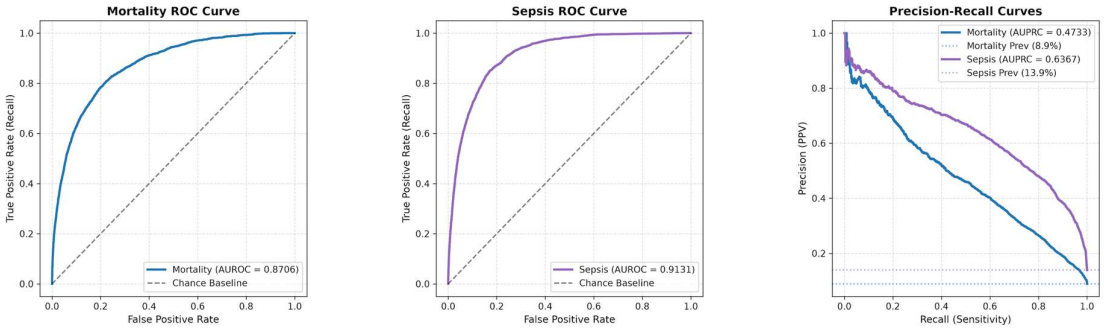


Fig 8.8: ROC curve and PR curve for mortality and sepsis

Table 8.6: GCN Evaluation Metrics

Metric	**Mortality Prediction**	**Sepsis Prediction**
Optimal Threshold	0.6300	0.4600
AUROC	0.8706	0.9131
AUPRC	0.4733	0.6367
Accuracy (%)	86.8800	79.5500
Balanced Accuracy (%)	76.92	83.2400
Sensitivity / Recall (%)	64.79	88.3400
Specificity (%)	89.04	78.1300
Precision (%)	36.63	39.5200
F1-Score	0.4680	0.5461
F2-Score	0.5616	0.7084

8.6.2. Total and Task Specific Loss

The total training and validation losses decreased throughout training. This indicates that the GCN progressively learned from the patient similarity graph. The total training loss declined from 2.43 to 1.75 and validation loss decreased from 2.45 to 1.84. Both curves flattened toward the later epochs. The task-specific losses show different patterns for mortality and sepsis. The mortality training loss decreased, reaching less than 0.7. Its validation loss stabilized around 0.82. This gap indicates some level of overfitting. The sepsis training and validation losses

closely aligned and converged near 1.01. Overall, the loss curves suggest stable optimization. Although, the continuing gap in mortality loss indicates that improvements in the training data did not translate fully to validation data according to the graphs below.

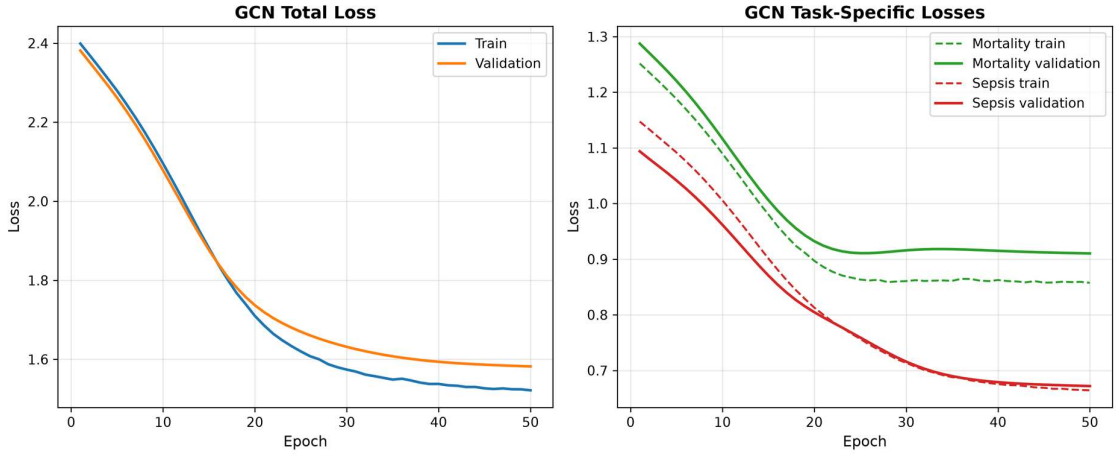


Fig 8.9: GCN Total and Task Specific Loss of GCN

8.6.3. Validation Performance

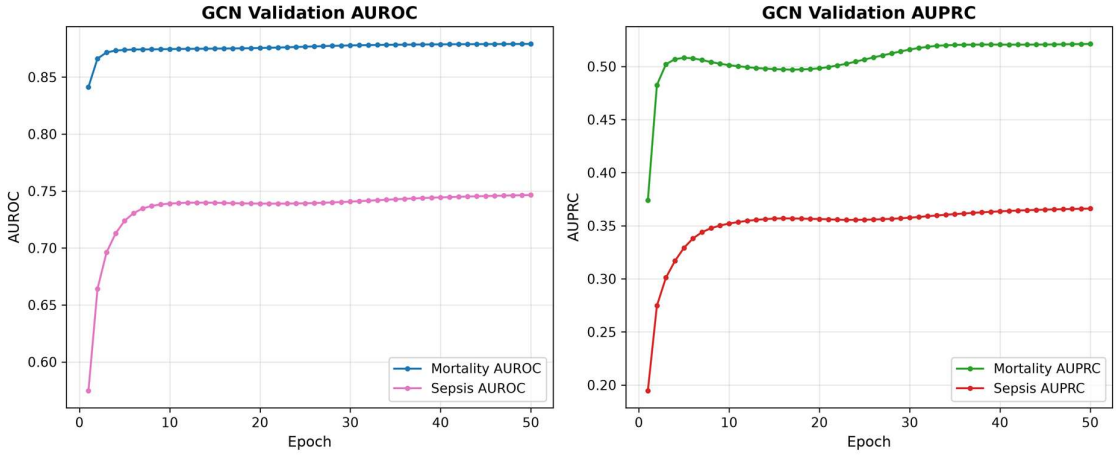


Fig 8.10: Validation Performance of GCN

The validation AUROC curves show that GCN learned to discriminate for both prediction tasks. A mortality AUROC starts from 0.83 and improves till 0.87 around epoch 4 and then becomes constant. Sepsis AUROC started to stop improving and reached 0.75. The validation AUPRC stopped improving around epoch 10 for sepsis and 4 for mortality. The figure shows signs of slight overfitting according to the validation AUPRC.

8.7. Drop in GCN

The metrics AUROC and AUPRC was measured along each stage and the results were found to improve in each stage. Mortality metrics improved slightly but sepsis metrics increased

significantly in stage 4 where GCN was implemented. These were the results found at every stage:

Table 8.7: Mortality and Sepsis AUROC and AUPRC at Different Stages

Stage	**Mortality AUROC**	**Mortality AUPRC**	**Sepsis AUROC**	**Sepsis AUPRC**
Stage 1: Temporal Encoder	0.8474	0.4590	0.8668	0.6105
Stage 2: Tabular Encoder	0.8350	0.4410	0.7850	0.4120
Stage 3: Attention Fused	0.8793	0.5293	0.8850	0.5840
Stage 4: GCN	0.8706	0.4733	0.9131	0.6367

There is a drop in mortality AUPRC in stage 4 which implements GCN. Even though Sepsis metrics surge a dramatic increase, there is mainly two reasons for this. Homophily is the tendency of connected nodes in a graph to have similar characteristics. Sepsis has high homophily. In feature space, septic patients cluster together and k = 5 acts as a great graph input for GCN. The hospital mortality has low homophily. Since mortality is not a single disease, two patients can have similar vitals with different mortality results. Multitask Loss Competition can also be one of the reasons. Even though the LSTM was optimized strictly for mortality predictions, the GCN must minimize joint loss.

SECTION 9: CONCLUSION AND FUTURE SCOPE

The project investigated the use of Long Short-Term Memory and Gated Recurrent Unit networks, Graph Neural Networks for predicting hospital mortality and Sepsis classification among ill patients using eICU–CRD. Initially, the tabular and temporal encoder got an AUROC of 0.8350 and 0.8474 respectively, after which multi-head attention fusion got an AUROC of 0.8793 for the same and finally GNN achieved a AUROC of 0.8706, displaying continuous improvement in the score for mortality prediction. Similarly, for sepsis prediction temporal encoder, tabular encoder, multi-head attention infusion, and the GNN got a score of 0.8668, 0.7850, 0.8850, and 0.9131 respectively. Mortality recall was 64.79%, while sepsis recall reached 88.34%. These results demonstrate different performance characteristics across the two tasks with stronger discrimination and recall for sepsis than for mortality.

The current work can be extended into a more comprehensive clinical prediction framework by adding more data modalities and a privacy preserving distributed learning. Federated learning framework would provide privacy for each hospital's data. Instead of transferring raw clinical data to a central server, hospitals would share model parameters. Federated averaging (FedAvg) could be implemented initially, followed by personalized or adaptive aggregation methods. In a healthcare setting, professionals require concrete explanations so deep learning models can produce ambiguity because they are a black box and it is difficult to interpret how decisions are made. Explainability methods, including GNN Explainer and attention visualization, could be used to investigate which features, modalities, or patient relationships influence predictions.

REFERENCES

[1] B. E. Keuning et al., “Mortality prediction models in the adult critically ill: A scoping review,” *Acta Anaesthesiologica Scandinavica*, vol. 64, no. 4, pp. 424–442, 2020, doi: 10.1111/aas.13527.

[2] H. Sun, M. Kang, H. Zhang, J. Jia, and Q. Wang, “Machine Learning for Predicting Mortality in Intensive Care Unit Patients: A Prognostic Performance Systematic Review and Meta-Analysis,” *Nursing in Critical Care*, vol. 30, no. 6, art. no. e70206, 2025, doi: 10.1111/nicc.70206.

[3] T. J. Pollard et al., “The eICU Collaborative Research Database, a freely available multi-center database for critical care research,” *Scientific Data*, vol. 5, art. no. 180178, 2018, doi: 10.1038/sdata.2018.178.

[4] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[5] K. Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014, pp. 1724–1734, doi: 10.3115/v1/D14-1179.

[6] Z. Che, S. Purushotham, K. Cho, D. Sontag, and Y. Liu, “Recurrent Neural Networks for Multivariate Time Series with Missing Values,” *Scientific Reports*, vol. 8, art. no. 6085, 2018, doi: 10.1038/s41598-018-24271-9.

[7] C. Wang, X. Yang, M. Sun, Y. Gu, J. Niu, and W. Zhang, “Multimodal fusion network for ICU patient outcome prediction,” *Neural Networks*, vol. 180, art. no. 106672, 2024, doi: 10.1016/j.neunet.2024.106672.

[8] Y. Wang and W. Li, “Integrating Multimodal EHR Data for Mortality Prediction in ICU Sepsis Patients,” *Statistics in Medicine*, vol. 44, art. no. e70060, 2025, doi: 10.1002/sim.70060.

[9] N. Nikravangolsefid et al., “Machine learning for predicting mortality in adult critically ill patients with Sepsis: A systematic review,” *Journal of Critical Care*, vol. 84, art. no. 154889, 2024, doi: 10.1016/j.jcrc.2024.154889.

[10] T. N. Kipf and M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” in *International Conference on Learning Representations (ICLR)*, 2017.

[11] M. Ma, P. Sun, Y. Li, and W. Huo, “Predicting the risk of mortality in ICU patients based on dynamic graph attention network of patient similarity,” *Mathematical Biosciences and Engineering*, vol. 20, no. 8, pp. 15326–15344, 2023, doi: 10.3934/mbe.2023685.

[12] T. Cover and P. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[13] R. Caruana, “Multitask Learning,” *Machine Learning*, vol. 28, no. 1, pp. 41–75, 1997.

[14] M. Sokolova and G. Lapalme, “A systematic analysis of performance measures for classification tasks,” *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.

[15] M. W. Heymans and J. W. R. Twisk, “Handling missing data in clinical research,” *Journal of Clinical Epidemiology*, vol. 151, pp. 185–188, 2022, doi: 10.1016/j.jclinepi.2022.08.016.

[16] A. Awad, M. Bader-El-Den, J. McNicholas, J. Briggs, and Y. El-Sonbaty, “Predicting hospital mortality for intensive care unit patients: Time-series analysis,” *Health Informatics Journal*, vol. 26, no. 2, pp. 1043–1059, 2020, doi: 10.1177/1460458219850323.

[17] S. Zhao, G. Tang, P. Liu, Q. Wang, G. Li, and Z. Ding, “Improving Mortality Risk Prediction with Routine Clinical Data: A Practical Machine Learning Model Based on eICU Patients,” *International Journal of General Medicine*, vol. 16, pp. 3151–3161, 2023, doi: 10.2147/IJGM.S391423.

[18] D. Shillan, J. A. C. Sterne, A. Champneys, and B. Gibbison, “Use of machine learning to analyse routinely collected intensive care unit data: A systematic review,” *Critical Care*, vol. 23, art. no. 284, 2019, doi: 10.1186/s13054-019-2564-9.

[19] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011.

[20] A. E. W. Johnson et al., “MIMIC-III, a freely accessible critical care database,” *Scientific Data*, vol. 3, art. no. 160035, 2016, doi: 10.1038/sdata.2016.35.

[21] A. E. W. Johnson et al., “MIMIC-IV, a freely accessible electronic health record dataset,” *Scientific Data*, vol. 10, art. no. 1, 2023, doi: 10.1038/s41597-022-01899-x.

[22] P. J. Thoral et al., “Sharing ICU Patient Data Responsibly Under the Society of Critical Care Medicine/European Society of Intensive Care Medicine Joint Data Science Collaboration: The Amsterdam University Medical Centers Database (AmsterdamUMCdb) Example,” *Critical Care Medicine*, vol. 49, no. 6, pp. e563–e577, 2021, doi: 10.1097/CCM.0000000000004916.

[23] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.

[24] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001.

[25] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[26] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 51–56.

[27] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[28] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[29] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” arXiv:1412.6980, 2014.

[30] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” in Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019, pp. 2623–2631, doi: 10.1145/3292500.3330701.

[31] A. Vaswani et al., “Attention is All You Need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[32] Q. Li, Z. Han, and X.-M. Wu, “Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning,” in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32, no. 1, 2018, doi: 10.1609/aaai.v32i1.11604.

[33] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[34] C. Elkan, “The foundations of cost-sensitive learning,” in *Proceedings of the 17th International Joint Conference on Artificial Intelligence*, 2001, pp. 973–978.

PLAG REPORT (Through Turnitin)Similarity Plagiarism Report + AI Plagiarism Report